



SDC26

Technical Report

Team ToBeDefined

Swarm Drone Challenge 2026 — ILA Berlin, 11 June 2026

Abstract

Team ToBeDefined flies a swarm of Parrot Anafi drones in the Swarm Drone Challenge 2026. Every drone runs the same software — on the real flight controller and in the Parrot Sphinx simulator — so we test in simulation exactly what we fly. Positioning is vision-only (ArUco markers, no GPS), manoeuvres are written in a small mission-scripting language, and a central command-and-control overlay coordinates the swarm. One rule runs through the whole design: move by what the camera can see, in small bounded steps, rather than steering to absolute coordinates we cannot trust indoors. The full stack — about 90,000 lines of Python — is open-source.

1. Introduction

The Swarm Drone Challenge 2026 is an indoor capture-the-flag for drones. Two teams of up to five Parrot Anafi share a 20 m × 10 m arena. Each team defends three target boxes and tries to flip the three enemy boxes by hovering over them — within a 1–2 m band — for at least two seconds. The boxes carry ArUco markers (DICT_4X4_50) whose colour flips on capture (leading digit 4 = red, 3 = blue; trailing digit is the box, 1–6). Points come in tiers:

- **1 point** — a plain capture.
- **5 points** — a capture with the whole team already outside its home zone.
- **10 points** — two captures by two drones within one second.
- **Instant win** — hold all six boxes for five seconds.

A match lasts ten minutes, and the hall has no GPS — so every position comes from the on-board cameras and inertial sensors.

2. System Architecture

Key idea Three small services share one code base: a flight controller per drone, a fleet hub, and a per-team strategy brain — all plain HTTP/JSON.

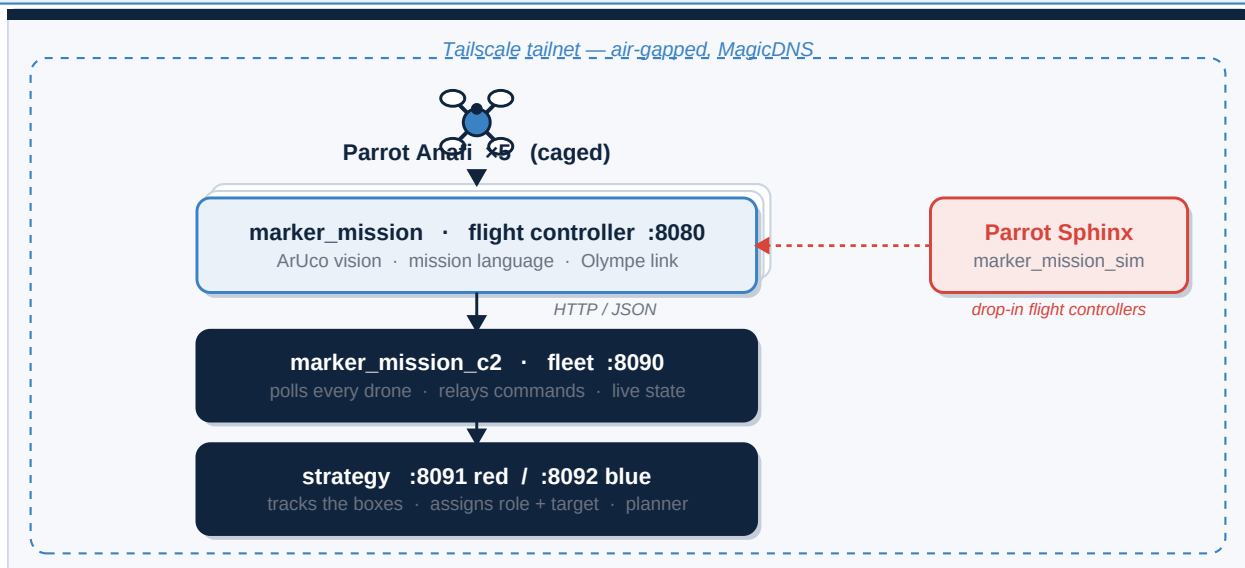


Figure 1 — The runtime. One flight controller per caged Anafi; the C2 aggregates the fleet; the per-team strategy servers decide each drone's job. The Sphinx simulator drops in for the real controllers during development.

- **marker_mission** (:8080), one per drone — the ArUco vision loop, the mission language, and the Olympe link to the Anafi.
- **marker_mission_c2** (:8090) — polls every drone, relays operator commands, and shows live state.
- **strategy** (:8091 red / :8092 blue) — tracks the six boxes and assigns each drone a role and a target.

The same code drives the Parrot Sphinx simulator, so a change is proven in simulation before it flies. One operator switch re-points a drone from the simulator to a real controller by IP; another hot-swaps the whole stack between the competition arena and a smaller test arena — live, with no restart.

Manoeuvres are written in a small text language. Its commands fall into four families:

- **Bounded moves** (FB_IMU, LR_IMU, UD_IMU, YAW_IMU) — go an exact distance or angle on the drone's own sensors, then stop.
- **Heading hold** (YAW) — face a fixed arena direction, so a move never inherits a stale heading.

- **Vision homing** (APPROACH, FB_BRAKE) — fly toward a named marker by sight; drift-free, no matter the world-frame fix.
- **Stick windows & holds** (FB_RC, UD_RC, HOOVER) — push a stick for a set time, or hover in place.

3. Localisation

Key idea Two cameras and a compass turn wall markers into a position — and we throw away any fix the sensors disagree on.

There is no GPS, so each drone works out where it is from the ArUco markers on the arena walls: eight pillars, sixteen markers (one at 2 m and one at 4 m on each), 0.5 m squares. The vision loop detects them and solves each marker's pose (Figure 2).

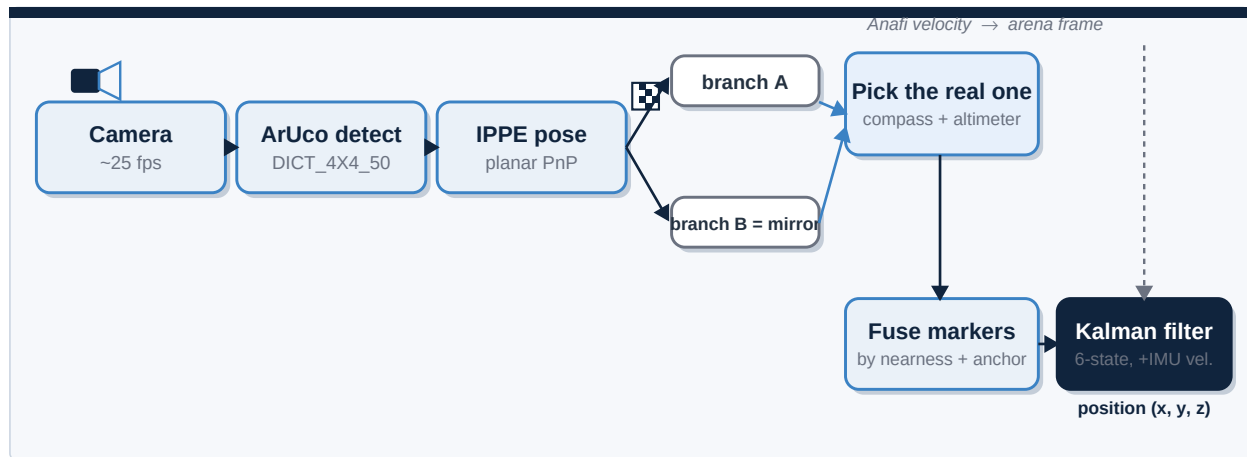


Figure 2 — From camera to position. The pose solver gives two mirror-image answers; a compass and an altimeter pick the real one; nearby markers are fused around an anchor and smoothed by a six-state Kalman filter that also reads the Anafi's velocity.

3.1 Resolving the mirror ambiguity

A single marker's pose solver returns two answers that are mirror images of each other — and only one is real (the other puts the drone on the far side of the marker). We pick the right one with two independent checks:

- **Compass** — keep the branch whose implied heading matches the magnetometer (within about 12°). This needs no altitude, so take-off starts from a clean fix.
- **Altimeter** — once airborne, the Anafi's downward ultrasound confirms the height of the chosen branch.

If neither check agrees and only one marker is in view, we reject the fix and let the last good estimate fade — better than jumping to a mirrored pose.

3.2 Smoothing and multiple markers

A six-state Kalman filter — position and velocity, $[p_x, p_y, p_z, v_x, v_y, v_z]$ — blends the slow but accurate marker fixes with the Anafi's fast velocity telemetry (rotated into the arena frame using the calibrated magnetic-north offset). When several markers are visible, their positions are averaged by nearness and tied to a remembered *anchor*, so one noisy reading cannot drag the estimate across the field. The filter also supplies the velocity the controller uses to damp side-to-side sway while homing.

3.3 No GPS

No GPS is used in flight — only as off-line ground truth in the simulator, per §4.4. And we never steer open-loop on an absolute fix: every precise move homes on a marker the camera can actually see (Section 4), which stays correct no matter which mirror the estimator picked.

4. Flight Control and Trajectory Planning

Key idea Every precise move is bounded and vision-relative — never an open-loop dash across the field.

The Anafi accepts both continuous stick commands and exact “move this far and stop” commands; we combine them with vision homing as building blocks (Section 2). Every attack is generated automatically for the chosen target and follows one pattern (Figure 3):

- Climb above the 0.73 m boxes.
- APPROACH the enemy marker by sight, stopping 1 m short.
- Rise into the 1–2 m capture band.
- One bounded FB_IMU step over the box centre.
- Hover at least two seconds — the marker flips.
- Climb, turn, and APPROACH our own wall marker to return.

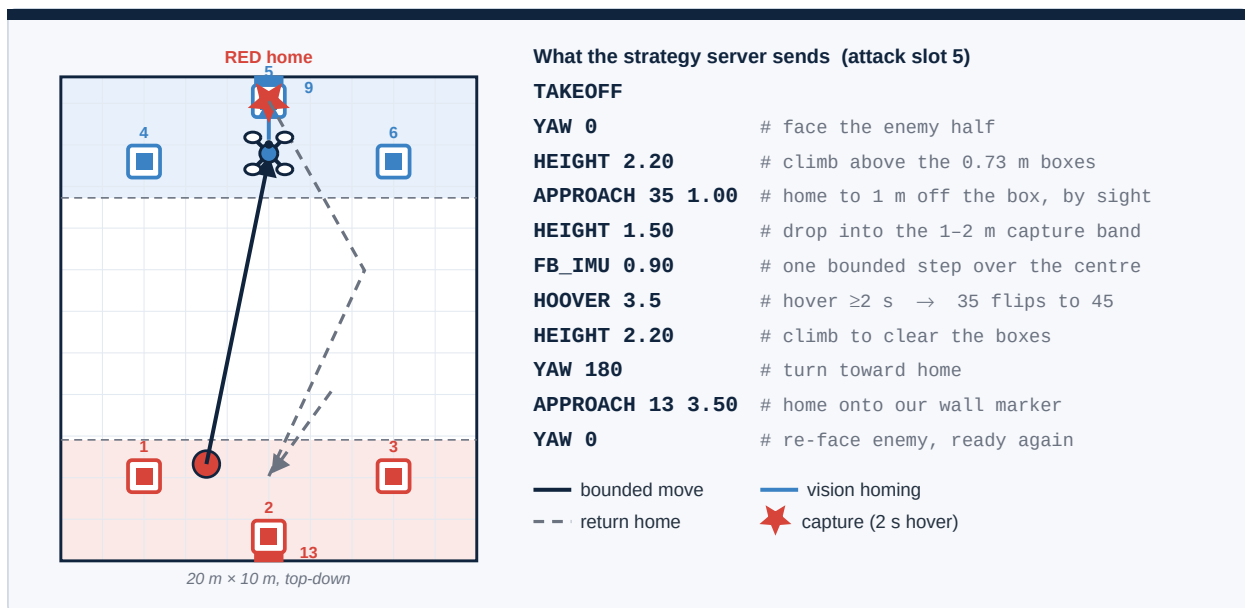


Figure 3 — The attack manoeuvre (left) and the exact script the strategy server emits (right). Navy = bounded move, blue = vision homing, dashed = vision-homed return, star = the 2-second capture hover.

Three lessons shaped this pattern:

- **No open-loop cruises.** Pitching forward tilts the camera down and hides the target marker, so a vision brake never fires and the drone runs to its timeout at full stick. Bounded steps and vision homing avoid this entirely.
- **A bounded step over the box.** Sized to the 1 m stand-off, so even a noisy fix stops over the target rather than carrying on into the wall behind it.
- **Closed-loop turns.** YAW/YAW_IMU hold an exact heading; a yaw-rate stick drifted in testing and sent the next leg sideways.

4.1 Safety

- A hard altitude ceiling clamps the throttle channel.
- A watchdog auto-lands a drone if the operator link drops for more than two seconds (§3.3).
- A single key-press lands the whole fleet — the Safety Officer's kill-all (§2.2).
- Every drone flies inside a protective cage (§3.2).
- The simulator enforces the same arena walls, so a script that would hit one is caught before it ever flies.

5. Swarm Strategy

Key idea A simple priority ladder, re-decided every tick — defend first, then go for points.

A small, rule-based planner runs on the strategy server every tick. It reads each drone (link, battery, position, phase) and the live box map, then hands every drone a role and a target. There is no learned policy — the rules are fixed and easy to follow, which matters because the code is open (Figure 4). The first rule that applies wins:

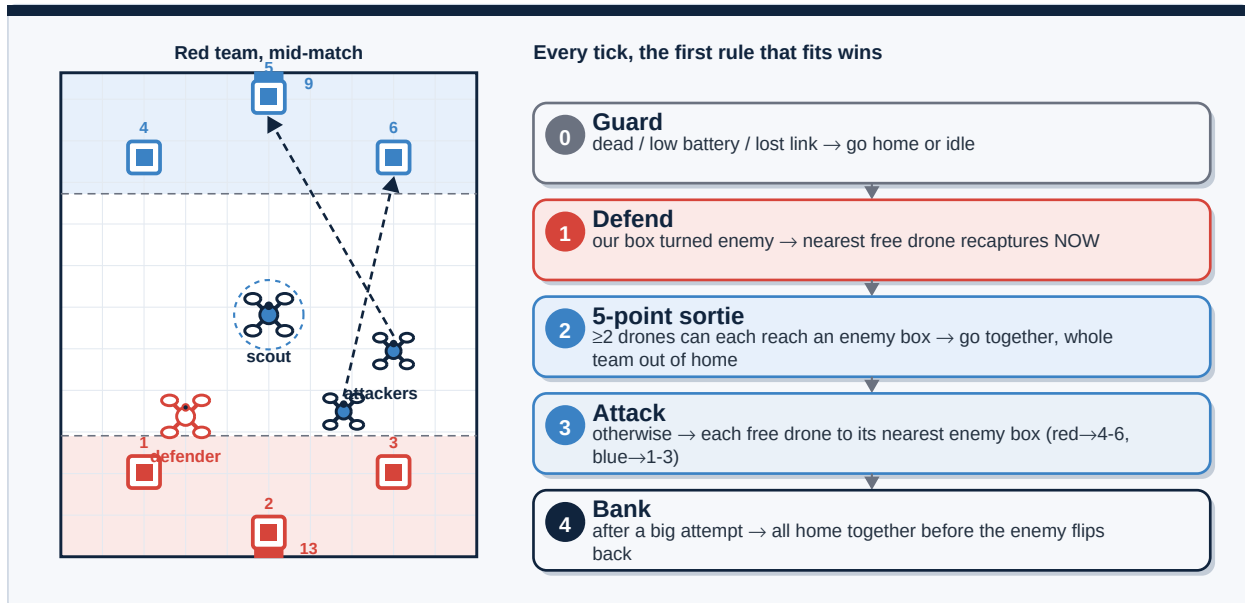


Figure 4 — Left: a typical mix — a scout refreshing the box map from the centre, two attackers vectored at enemy boxes, a defender holding in the neutral zone. Right: the per-tick priority ladder.

A recapture inside our own home zone scores nothing (§1.4.3), so the planner treats it as defence, not points. Holding all six boxes for five seconds is the instant win and overrides everything when it is within reach.

5.1 Roles and fair play

- **Four roles:** attacker, scout (spins at the centre to keep the box map fresh), defender, and idle/return.
- **No double-booking.** Three running sets — slots already taken this tick, slots an in-flight drone is already heading to, and slots inside the 5-second post-capture lock — stop two drones chasing the same box.
- **Asymmetric attack.** Red may target only boxes 4–6, blue only 1–3; the planner refuses any operator override that breaks this.

5.2 The defender

The defender never camps over a box — that risks the dead-drone-over-target rule and forfeits the coordination bonus. Instead it hovers just inside the neutral zone, facing its own back wall and holding station by sight on the back-wall marker. Because the box map is shared, it sees a loss the instant a marker flips, darts in to recapture, then returns to station.

5.3 Open source

Everything is open at github.com/otiedemann/sdc-tobe — about 90,000 lines of Python across the flight controller, the C2, the strategy layer, and the simulator, sharing one configuration and one environment. Every flight is logged tick-by-tick and recorded to video for an in-browser replay viewer, so any result in this report can be reproduced.

6. Conclusion

One idea runs through the whole stack: move by what the camera can see, in small bounded steps, instead of steering to coordinates we cannot trust indoors. That is what keeps the swarm flying safely and repeatably. What remains — a sanity check that lets the active wall guard switch back on once a fix is trusted, and more tuning of the coordinated five-point sortie — is scoped for the final at ILA Berlin on 11 June 2026.